

BÁO CÁO ĐỒ ÁN MÔN HỌC

(Đồ án phát triển ứng dụng)

Lớp: IT003.Q21.TTNT

SINH VIÊN THỰC HIỆN: LÊ GIA HUY

Mã sinh viên: 25520693

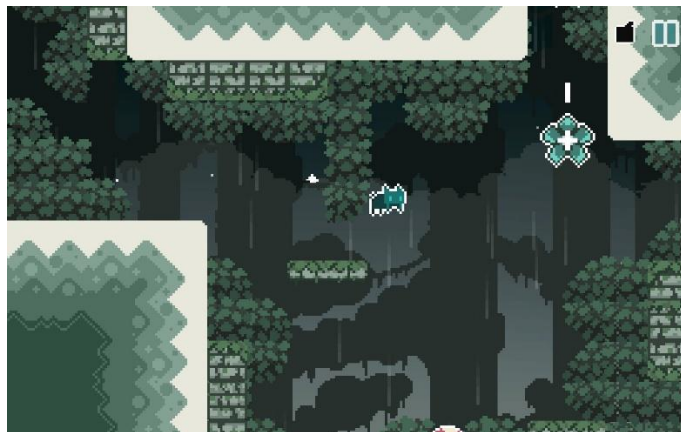
TÊN ĐỀ TÀI: THỰC HIỆN MÔ PHỎNG GAME 2D PLATFORMER ĐƠN GIẢN (LẤY CẢM HỨNG TỪ *NAMELESS CAT*)

CÁC NỘI DUNG CẦN BÁO CÁO:

1. Giới thiệu đồ án:

a. Mô tả chung về ứng dụng:

- Là một tựa game 2D Platformer (đi cảnh vượt chướng ngại vật) được phát triển hoàn toàn bằng ngôn ngữ Python, sử dụng nền tảng thư viện Pygame. Trò chơi được lấy cảm hứng từ tựa game nổi tiếng "Nameless Cat".



(Hình ảnh minh họa game Nameless Cat)

Nguồn: Apkpure.com

b. Các CTDL và giải thuật đã được sử dụng:

*Cấu trúc dữ liệu:

- **Bảng băm (Hash table/dict):** Bảng băm được sử dụng trong đồ án để phục vụ một số nhiệm vụ nhất định
 - Ứng dụng kỹ thuật **Spatial Partitioning** bằng cấu trúc dữ liệu dict để lưu các ô có chứa block với key là một tuple tọa độ (x, y).
 - **Mục đích:** Tối ưu hóa thuật toán AABB cho chương trình bằng cách loại bỏ hiện tượng thắt cổ chai (bottleneck) CPU khi nội suy va chạm vật lý, tối ưu hóa triệt để tài nguyên cho game 2D. Các truy xét va chạm tận dụng tối đa tốc độ truy cứu $O(1)$ của CTDL dựa trên key là các tọa độ (x, y).
 - Quản lý trạng thái Animations: cấu trúc dữ liệu dict được sử dụng để hỗ trợ lưu trữ các trạng thái khác nhau của một entity. Giúp chuyển đổi qua lại giữa các trạng thái.

*Thuật toán cốt lõi:

- **Thuật toán Kẹp biên (Hard Clamping algorithm):**
 - **Khái niệm:** Kẹp biên là một kỹ thuật toán học dùng để giới hạn một giá trị x nằm trong khoảng an toàn từ *MIN* đến *MAX*.
 - **Nguyên lý hoạt động:** Thuật toán hoạt động dựa trên việc kết hợp lồng nhau hai hàm tìm giá trị nhỏ nhất và giá trị lớn nhất)
$$f(x) = \max(\text{MIN}, \min(x, \text{MAX}))$$
 - **Độ phức tạp:**
 - Độ phức tạp thời gian: $O(1)$. Thuật toán sử dụng đúng 2 phép toán so sánh cơ bản, min và max ở tầng vi xử lý, không sử dụng vòng lặp.
 - Độ phức tạp không gian. $O(1)$. Thuật toán không tốn thêm bất kỳ vùng nhớ nào để lưu trữ dữ liệu phụ trợ.

- **Các thuật toán tương tự:**

- Sử dụng vòng lặp if, else để tái tạo logic tương tự.
- Sử dụng thuật toán Soft Clamping: Kéo camera về biên một cách từ từ bằng nội suy tuyến tính, tạo hiệu ứng “nảy” khi đụng tường thay vì dừng cứng

- ➔ **Lý do chọn thuật toán Hard Clamping:**

- Gộp toàn bộ logic của khối lệnh if, else thành 1 dòng code hiệu quả hơn.
- Đối với một tựa game platformer pixel-art, việc Camera dừng cứng (Hard Stop) ở mép màn hình mang lại cảm giác phù hợp và chuyên nghiệp hơn vì việc tạo motion camera, muốn nhìn ra khỏi map là không cần thiết.

- **Thuật toán Phân giải Va chạm AABB (Axis-Aligned Bounding Box):**

- **Khái niệm:** Xét va chạm (collisions) giữa các object thực thể bằng một hitbox hình chữ nhật bao quanh chính các object đó.
- **Nguyên lý hoạt động:** Thuật toán so sánh tọa độ của 2 hình chữ nhật dựa trên phép chiếu hình.
 - Hình A và Hình B chỉ xảy ra va chạm khi và chỉ khi: **Hình chiếu của chúng trên trục X giao nhau VÀ Hình chiếu trên trục Y cũng giao nhau.**
 - Dưới góc độ lập trình, nó chỉ là 4 phép so sánh logic AND cực kỳ đơn giản: Cạnh phải A > Cạnh trái B, Cạnh trái A < Cạnh phải B, Cạnh dưới A > Cạnh trên B, Cạnh trên A < Cạnh dưới B.
- **Độ phức tạp:**
 - Độ phức tạp thời gian: $O(1)$. **Cấp độ vi mô:** Việc xét va chạm giữa 2 vật thể có độ phức tạp là $O(1)$ vì chỉ dùng 4 phép so sánh cộng trừ.

- Độ phức tạp thời gian: $O(1)$. **Cấp độ vĩ mô:** Nếu đem E thực thể đi check với N khối gạch trên bản đồ theo cách duyệt mảng thông thường, độ phức tạp sẽ là $O(E \times N)$.
- Độ phức tạp không gian: $O(1)$. Thuật toán lấy trực tiếp tọa độ bộ nhớ có sẵn để tính toán, không sinh ra thêm mảng phụ trợ nào.
- **Các thuật toán tương tự:**
 - **Pixel-Perfect Collision (Va chạm điểm ảnh):** Cực kỳ chính xác. Máy tính sẽ đếm từng điểm ảnh (pixel) không trong suốt của Mèo và Quái xem có đè lên nhau không.
 - *Nhược điểm:* Độ phức tạp lên tới $O(W \times H)$ (W, H là kích thước ảnh). Quá nặng và gây lãng phí tài nguyên khủng khiếp.
 - **Circle Collision (Va chạm đường tròn):** Tính khoảng cách giữa 2 tâm đường tròn bằng định lý Pytago rồi so sánh với tổng 2 bán kính.
 - *Nhược điểm:* Rất nhanh, nhưng hình tròn không phù hợp với phong cách của tựa game (pixel, khối vuông). Có thể gây ra các hitbox ảo khó xử lý.
 - **SAT (Separating Axis Theorem):** Thuật toán tối thượng dùng để xét va chạm cho các **đa giác có khả năng xoay nghiêng**.
 - *Nhược điểm:* Toán học Vector rất phức tạp, tốn CPU quá phức tạp tại một tựa game mà các object chắc chắn chỉ có ở dạng hình chữ nhật.
- ➔ **Lý do chọn thuật toán AABB:**
 - **Tương thích tuyệt đối với kiến trúc Tilemap:** Bản đồ được xây dựng từ phần mềm LDK theo dạng lưới vuông (Grid-based). Việc dùng các hộp AABB vuông vức ôm sát vào các lưới này là sự lựa chọn hoàn hảo về mặt hình học.

- **Tối ưu hóa năng lực vi xử lý (CPU Performance):** Thay vì bắt CPU phải tính căn bậc hai (như Circle) hay phân tích điểm ảnh (như Pixel-Perfect), AABB chỉ ép CPU làm phép tính của học sinh lớp 1 (Cộng, Trừ, Lớn hơn, Nhỏ hơn). Điều này giúp game chạy tốt ở 60 FPS kể cả trên các máy tính cấu hình yếu.
- **Độ chính xác "Vừa đủ" (Good-enough Precision):** Đối với dòng game Platformer cổ điển, người chơi không cần độ chính xác đến từng chi tiết. Một khối hộp AABB ôm sát hình ảnh (được tinh chỉnh qua `get_bounding_rect()`) đã tạo ra một trải nghiệm vật lý vô cùng chặt chẽ và đáng tin cậy.

- Thuật toán Chuẩn hóa Vector & Nội suy góc xoay (Vector Normalization & `math.atan2`)

- **Khái niệm:** Đây là bộ đôi thuật toán toán học dùng để tính toán quỹ đạo bay của đạn bắn trúng mục tiêu đang di chuyển, đồng thời xoay hình ảnh (sprite) của viên đạn luôn hướng mũi về phía trước.
- **Nguyên lý hoạt động:** Thuật toán chia làm 2 giai đoạn:
 - **Giai đoạn 1 - Chuẩn hóa Vector (Vector Normalization):**
 - Đầu tiên, tính khoảng cách tuyệt đối giữa Quái vật và Mèo bằng định lý Pytago: $distance = \sqrt{dx^2 + dy^2}$ (thông qua hàm `math.hypot`).
 - Sau đó, chia từng thành phần khoảng cách (`dx`, `dy`) cho tổng `distance` để tạo ra một **Vector đơn vị (Unit Vector)** có độ dài luôn bằng 1.
 - Cuối cùng, nhân Vector đơn vị này với tốc độ đạn (`bullet_speed`). Viên đạn sẽ bay thẳng đến mục tiêu với một vận tốc không đổi, bất chấp việc mục tiêu đứng sát bên hay ở tít góc màn hình.
 - **Giai đoạn 2 - Nội suy góc xoay (`atan2`):**

- Để viên đạn (như con dơi) không bị bay lùi hoặc bay ngang đơ cứng, thuật toán dùng hàm Arctangent 2 tham số:

$$\text{angle} = \text{atan2}(-dy, dx)$$
- Hàm này trả về góc lệch tính bằng Radian, sau đó được đổi sang Độ (Degrees) để xoay khung hình (Bounding box) của ảnh dơi hướng thẳng vào mục tiêu.
- **Độ phức tạp:**
 - **Độ phức tạp thời gian:** $O(1)$. Quá trình tính toán đường bay và góc xoay cho mỗi viên đạn chỉ bao gồm một vài phép toán vô hướng (scalar math) như khai căn, chia, và tính lượng giác. Các phép toán này được xử lý trực tiếp ở cấp độ thanh ghi của vi xử lý nên thời gian thực thi là hằng số.
 - **Độ phức tạp không gian:** $O(1)$. Thuật toán không khởi tạo thêm mảng hay ma trận nào, chỉ dùng một vài biến cục bộ (vx, vy, angle) lưu trữ trong Stack memory rồi giải phóng ngay.
- **Các thuật toán tương tự:**
 - **Thay thế Giai đoạn 1 (Tính khoảng cách):** Có thể dùng công thức Python thuần $\text{distance} = (\text{dx}^2 + \text{dy}^2)^{0.5}$. Tuy nhiên, cách này chạy chậm hơn và dễ gây ra lỗi tràn số (Overflow) nếu tọa độ cực kỳ lớn.
 - **Thay thế Giai đoạn 2 (Tính góc):** Có thể dùng hàm lượng giác cơ bản $\text{math.atan}(y/x)$.
 - *Nhược điểm:* Nếu Target nhảy thẳng đứng lên đầu entity (tức là $x=0$), phép chia $y/0$ sẽ khiến game văng lỗi (Crash). Hơn nữa, atan không xác định rõ hướng khiến viên đạn có thể bay ngược hướng.

➔ **Lý do chọn thuật toán:**

- **Đảm bảo tính nhất quán của tốc độ:** Nếu không chuẩn hóa Vector, viên đạn sẽ lấy thẳng khoảng cách dx, dy làm vận tốc. Khi Target đứng xa, dx lớn, đạn bay nhanh; khi Mèo đứng gần, đạn bay chậm. Chuẩn hóa Vector ép vận tốc về một hằng số duy nhất (bullet_speed = 7.0), giúp game công bằng và dễ dự đoán để người chơi có thể né tránh.

- Máy trạng thái hữu hạn (Finite State Machine - FSM)

- **Khái niệm:** Máy trạng thái hữu hạn (FSM) là một mô hình toán học tính toán, đồng thời là một mẫu thiết kế (Design Pattern) kinh điển trong lập trình AI Game. Nó quy định rằng một thực thể (Entity) chỉ có thể ở **duy nhất một trạng thái (State)** tại bất kỳ thời điểm nào, trong số một tập hợp hữu hạn các trạng thái cho trước.
- **Nguyên lý hoạt động:** FSM hoạt động dựa trên 3 thành phần lõi: *Trạng thái (States)*, *Điều kiện chuyển đổi (Transitions)*, và *Hành động (Actions)*.
 - Trong class Chase_Enemy, não bộ của quái vật được chia thành các trạng thái tách biệt: 'walk', 'chasing', 'attack', 'ignore'.
 - **Logic:** Khi ở trạng thái 'walk', nếu Cảm biến Tầm nhìn (Radar) phát hiện người chơi → Kích hoạt Transition → Chuyển sang 'chasing'. Khi khoảng cách va chạm bằng 0 → Chuyển sang 'attack'.
 - **Đặc tính Animation-driven State (Hoạt ảnh chi phối trạng thái):** Khi vào trạng thái 'attack', FSM lập tức khóa cứng biến `vel_x = 0`. Quái vật bị ép phải đứng im tại chỗ cho đến khi hệ thống đọc hết chiều dài của mảng hoạt ảnh chém (`len(self.animations['attack'])`), sau đó FSM mới tự động "nhả số" về lại trạng thái 'walk'.
- **Độ phức tạp:**
 - **Độ phức tạp thời gian (Time Complexity):** $O(1)$. Nhờ việc kết hợp FSM với cấu trúc dữ liệu Bảng băm (Dictionary) chứa hoạt ảnh, việc

máy tính kiểm tra trạng thái hiện tại và xuất hình ảnh tương ứng diễn ra ngay lập tức, không tốn bất kỳ chu kỳ vòng lặp nào.

- **Độ phức tạp không gian (Space Complexity):** $O(1)$. Hệ thống chỉ cần duy trì một biến chuỗi (String) duy nhất (như `self.state = 'attack'`) trong bộ nhớ RAM để làm cờ hiệu (Flag) điều hướng toàn bộ logic.
- **Các thuật toán tương tự:**
 - **Behavior Trees (Cây hành vi):** Một hệ thống AI module hóa hiện đại, dùng để thiết kế AI cho các game AAA 3D phức tạp (như Halo, The Last of Us) hoặc quái vật có khả năng phối hợp nhóm.
 - *Nhược điểm:* Quá nặng, quá phức tạp nếu áp dụng vào một tựa game 2D Pixel mà quái vật chỉ có từ 2 đến 4 hành động cơ bản.

➔ **Lý do chọn thuật toán:**

- **Triệt tiêu Xung đột Logic (Mutual Exclusion):** Đặc tính "chỉ ở một trạng thái tại một thời điểm" của FSM giúp giải quyết triệt để các lỗi logic (như AI vừa tấn công vừa đi lùi, hoặc vừa rớt xuống vực vừa rượt theo người chơi). Trạng thái nào sẽ chỉ chạy bộ luật vật lý của riêng trạng thái đó.
- **Sự đơn giản của thuật toán:** Tránh những sự phức tạp hóa không cần thiết đối với một bộ animations và trạng thái nhỏ.

- **Thuật toán Ánh xạ dữ liệu không gian (1D to 2D Array Mapping)**

- **Khái niệm:** Là một kỹ thuật chuyển đổi cấu trúc dữ liệu, biến một mảng 1 chiều liên tiếp (ví dụ: `[0, 1, 0, 1...]`) thành một lưới không gian 2 chiều có Hàng (Row) và Cột (Column).
- **Nguyên lý hoạt động:** Sử dụng toán học modulo (chia lấy dư) và phép chia nguyên (Integer division). Nếu biết trước chiều rộng của bản đồ là W (số lượng cột), với mỗi phần tử nằm ở vị trí thứ i trong mảng 1D, tọa độ 2D của nó được tính bằng công thức:

- **Tọa độ cột (Trục X):** $col = i \bmod W$
 - **Tọa độ hàng (Trục Y):** $row = \left\lfloor \frac{i}{W} \right\rfloor$
 - Sau khi tính toán được tọa độ (col, row), thuật toán nhân các giá trị này với kích thước của một khối gạch (TILE_SIZE = 32px) để dịch ngược ra tọa độ Pixel thực tế trên màn hình hiển thị.
 - **Độ phức tạp:**
 - **Độ phức tạp thời gian:** $O(N)$, với N là tổng số lượng Tile (ô gạch) trên toàn bản đồ. Thuật toán chỉ duyệt qua mảng 1D đúng một lần duy nhất lúc khởi tạo màn chơi (Load Map) nên hoàn toàn không ảnh hưởng đến tốc độ khung hình (FPS) trong lúc chơi.
 - **Độ phức tạp không gian:** $O(N)$ để phân bổ bộ nhớ cho cấu trúc dữ liệu mới. (Tuy nhiên, nhờ tích hợp thẳng vào Bảng băm cục bộ ở bước sau, những ô không khí mang giá trị 0 bị loại bỏ, giúp độ phức tạp không gian thực tế giảm xuống đáng kể).
 - **Các thuật toán tương tự:**
 - Có một giải pháp mang tính "đánh đổi" (Trade-off) thường được sử dụng ở các hệ thống nhúng có RAM siêu nhỏ:
 - **Giữ nguyên mảng 1D (Flattened 2D Array):** Thay vì tốn tài nguyên chuyển đổi mảng 1D thành 2D, hệ thống cứ giữ nguyên cái mảng 1D đó trong RAM. Mỗi khi con mèo muốn xét va chạm ở tọa độ (x, y) , máy tính sẽ tính ngược lại chỉ số 1D bằng công thức: $index = y * W + x$.
 - **Nhược điểm:** Việc này ép CPU phải thực hiện phép nhân và cộng liên tục trong mỗi vòng lặp 60 FPS, làm tăng tải xử lý logic vật lý.
- ➔ **Lý do chọn thuật toán:**

- **Tính trực quan (Intuitiveness):** Việc dịch mã JSON ra một ma trận 2D giúp mã nguồn trở nên rõ ràng, logic toán học khớp hoàn toàn với hệ trục tọa độ Oxy của màn hình.
- **Tiền đề cho kiến trúc Spatial Hashing:** Trong quá trình ánh xạ, thuật toán đóng vai trò như một màng lọc: nó quét qua mảng 1D, xác định tọa độ lưới, rồi lập tức “lọc” những ô là đất (giá trị 1) cho vào Bảng băm không gian (Dictionary spatial_platforms), đồng thời vứt bỏ toàn bộ ô không khí (giá trị 0).

*CÁC KỸ THUẬT LẬP TRÌNH ỨNG DỤNG:

- **Tạo chuyển động bằng Toán học Lượng giác (Sinusoidal Movement):**
 - Thay vì sử dụng các hệ thống khung hình (Sprite Animation) phức tạp để tạo hiệu ứng bay lơ lửng cho Aiming_Enemy, đồ án đã sử dụng toán học lượng giác để điều hướng tọa độ trực tiếp, tiết kiệm RAM đồ họa.
 - **Nguyên lý hoạt động:** Trục Y của thực thể được cập nhật liên tục dựa trên hàm dao động điều hòa: $y = y_{orig} + \sin\left(\frac{time_now}{300}\right) \times 15$
 - **Giá trị kỹ thuật:** * Hàm sin giúp giá trị luôn dao động nhịp nhàng từ -1 đến 1. Nhân với hằng số biên độ (15), quái vật sẽ đập diu lên xuống trong khoảng 15 pixel.
 - Biến thời gian thực của hệ thống (pygame.time.get_ticks()) được chia cho một hệ số làm chậm (Time Scale = 300) để kéo giãn chu kỳ sóng, giúp chuyển động không bị giật cục mà vô cùng êm ái. Kỹ thuật này tạo ra chuyển động vĩnh cửu với độ phức tạp tính toán chỉ $O(1)$.
- **Kỹ thuật Cảm biến không gian hẹp (Edge Detection with Sensor_rect):**
 - Là phương pháp mô phỏng tầm nhìn vật lý hướng xuống đất của thực thể, giúp AI nhận biết được địa hình đứt gãy (vực thẳm) để kịp thời quay đầu.
 - **Nguyên lý hoạt động:** Thuật toán khởi tạo một hình chữ nhật ảo (sensor_rect) siêu hẹp (rộng 2px) và cực dài (100px), đóng vai trò như một "cọc dò đường" tàng hình.
 - Vị trí của cọc được đặt nhô ra phía trước hướng di chuyển của AI một khoảng cách an toàn (5px).

- Mỗi khung hình, hệ thống sẽ sử dụng thuật toán AABB để hỏi Bảng băm không gian (Spatial Hash): "*Cái cốc này có đang đâm trúng viên gạch nào bên dưới không?*".
- Nếu danh sách va chạm trả về là rỗng (False), AI kết luận phía trước là vực thẳm và lập tức kích hoạt cờ `facing_right = not facing_right` để đảo chiều di chuyển.
- **Độ phức tạp của thuật toán:**
 - **Độ phức tạp thời gian: O(1).** Vì cái cốc này là một hình chữ nhật, nó tích hợp hoàn hảo vào hệ thống va chạm AABB có sẵn. Việc kiểm tra chỉ tốn vài phép toán so sánh cục bộ.
 - **Độ phức tạp không gian: O(1).** Đối tượng `pygame.Rect` tàng hình này được tạo ra và thu gom rác (Garbage Collection) ngay trong cùng một khung hình, hầu như không chiếm dụng RAM.
- **Thuật toán thay thế:**
 - **Toán học Raycasting (Bắn tia laser):** Thay vì dùng hình chữ nhật, hệ thống bắn một tia vector tuyến tính từ mắt AI chếch xuống đất.
 - *Nhược điểm:* Đòi hỏi toán học vector phức tạp để tính toán điểm giao cắt (Intersection) với các cạnh của khối gạch, nặng hơn so với việc dùng `Rect` có sẵn của Pygame.
 - **Kiểm tra Index lưới tọa độ trực tiếp:** Chỉ cần quy đổi tọa độ X phía trước thành cột (`col`), và check xem ô (`col, row + 1`) có gạch không.
 - *Nhược điểm:* Rất thô cứng. Lỡ thiết kế bản đồ có một cái dốc hoặc bậc thang sâu 2 ô lưới, AI check ô `row + 1` không thấy gạch sẽ quay đầu ngay lập tức, dẫn đến việc quái vật không bao giờ biết đi xuống cầu thang.
- **Lý do lựa chọn kỹ thuật:**
 - **Khả năng thích ứng địa hình (Terrain Adaptability):** Chiều sâu 100px của chiếc cốc là một "con số vàng". Nó cho phép quái vật phân biệt được đâu là "vực thẳm thực sự" (sâu hơn 100px) và đâu chỉ là một "bậc thang đi xuống" (sâu khoảng 30px - 60px). Nếu là bậc thang, cốc vẫn chạm gạch và quái vật sẽ tự tin bước xuống.
 - **Tái sử dụng mã nguồn (Reusability):** Bằng cách dùng `sensor_rect`, em tái sử dụng được 100% logic hàm kiểm tra va chạm AABB đã viết trước đó mà không cần để ra thêm một hệ thống quy chiếu (Raycast) mới.
- **Tự động hóa Điều hướng (Wall Detection & Auto Jump):**
 - **Khái niệm:** Là thuật toán biến AI từ thể bị động (đụng tường thì quay đầu) sang thể chủ động phân tích môi trường để vượt qua chướng ngại vật nhỏ (Parkour).

- **Nguyên lý hoạt động:** Khi thuật toán AABB trục X phát hiện có sự va chạm với một khối gạch (Wall), thay vì quay đầu ngay lập tức, AI sẽ thực hiện 2 bước thẩm định:
 - **Kiểm tra độ cao (Jump Capacity):** Thuật toán tính toán `obstacle_height` bằng cách lấy tọa độ đáy chân của AI trừ đi tọa độ đỉnh của khối gạch. Nếu độ cao này lớn hơn 0 và nhỏ hơn sức bật tối đa (ví dụ 35px), khối gạch này "có thể nhảy qua".
 - **Kiểm tra khoảng không (Head Clearance):** AI tiếp tục truy vấn Bảng băm cục bộ xem ô lưới nằm ngay phía trên khối gạch đó có bị chặn hay không (để tránh trường hợp quái vật nhảy lên và kẹt đầu vào trần nhà).
 - Nếu thỏa mãn cả 2, AI bơm một lực vận tốc âm vào trục Y (`vel_y = jump_strength`), giả lập một cú bật nhảy hoàn hảo, đồng thời vẫn giữ nguyên tốc độ lao tới phía trước (`vel_x`).
- **Độ phức tạp:**
 - **Độ phức tạp thời gian: $O(1)$.** Chỉ tốn thêm 1 phép tính trừ tọa độ và 1 lần tra cứu Bảng băm (`col, row`) in `spatial_platforms`.
 - **Độ phức tạp không gian: $O(1)$.** Hoàn toàn xử lý bằng các biến cục bộ (Local variables).
- **Kỹ thuật thay thế:**
 - **Hệ thống Điểm mốc điều hướng (NavMesh / Waypoints):** Các game hiện đại thường bắt người thiết kế màn chơi (Level Designer) phải đặt các "khối tảng hình đánh dấu chỗ nhảy" trên bản đồ. Khi AI đụng khối tảng hình đó, nó tự động nhảy.
 - *Nhược điểm:* Cực kỳ tốn công sức thiết kế. Đòi bản đồ là phải đi đặt lại từng điểm nhảy một bằng tay.
 - **Thuật toán tìm đường A* (A-Star Pathfinding):** AI tự quy hoạch toàn bộ đường đi từ A đến B trước khi di chuyển.
 - *Nhược điểm:* Quá nặng và phức tạp cho một game platformer hành động nhanh.

Quá trình thực hiện

a. Tuần 1: Xây dựng kiến trúc lõi. Tổng quan của game

- Khởi tạo Git Repository và môi trường Pygame.
- Thiết lập Vòng lặp trò chơi (Game Loop) ở 60 FPS.
- Xây dựng hệ thống lớp (Class) chuẩn OOP cho Thực thể (Entity).
- Lập trình thuật toán Vật lý (Trọng lực, Nhảy) và Va chạm AABB.
- Tích hợp hệ thống Camera bám theo trục ngang.

b. Tuần 2: Xây dựng hệ thống Hoạt ảnh (Animations) và xử lý bản đồ

- Xây dựng thuật toán duyệt bản đồ

- Tích hợp file CSV: Đã hoàn thiện thuật toán đọc ma trận bản đồ từ file map.csv và hiển thị địa hình một cách tự động. Hiện tại, bản đồ vẫn đang ở mức thử nghiệm cơ bản, sử dụng ma trận nhị phân (0 và 1). Phần đồ họa bản đồ chi tiết hơn đang tiếp tục được thiết kế và hoàn thiện.

- Xây dựng hệ thống quản lý hình ảnh và hoạt ảnh của nhân vật chính (main player):

- Đã khắc phục hiện tượng “hitbox ảo” phát sinh do phần khoảng trắng trong suốt (transparent padding) của file ảnh gốc bằng cách tích hợp và sử dụng hàm “get_bounding_rect()” có sẵn trong thư viện **Pygame** nhằm xác định chính xác vùng va chạm của nhân vật.
- Quản lý bộ nhớ đồ họa: Xây dựng module trích xuất khung hình (Frame Extraction) 32x32 pixel kết hợp kỹ thuật subsurface và sao chép sâu (deep copy), loại bỏ triệt để hiện tượng rác dữ liệu ảnh (visual artifacts).
- Máy trạng thái hữu hạn (FSM): Triển khai vòng lặp hoạt ảnh hỗ trợ đa tốc độ cấu hình động (Dynamic Speed Dictionary) và phản chiếu đồ họa theo hướng di chuyển (Sprite Flipping).

c. Tuần 3: Xây dựng hệ thống Hoạt ảnh (Animations) và xử lý bản đồ

- Cải thiện Lớp Enemy_Base:

- Cải tiến Class Enemy_Base để phù hợp với tính kế thừa và tái sử dụng của code. Tạo tiền đề để tạo ra các Enemy đa dạng hơn cho trò chơi.

- Hoàn thiện máy trạng thái Game

- Thử nghiệm thành công bộ điều phối luồng game với 3 trạng thái độc lập: *Start_menu*, *Playing*, *Game_over*.

- Cải thiện Core Physics:

- Cải thiện thuật toán AABB với tính năng xử lý va chạm theo phương ngang, đảm bảo các Entity và nhân vật không bị đi xuyên tường.

- **Tích hợp UI Assets và xử lý đồ họa đa bộ đệm:**
 - Xây dựng thử nghiệm Lớp ImageButton, ứng dụng thuật toán cắt ảnh từ sheet.
- ➔ **Nhận xét:** Nhìn chung, tuần 3 đang dần hoàn thiện các tính năng cuối cùng của trò chơi. Tuần cuối cùng sẽ là tuần đầu tư về đồ họa và các tính năng phụ giúp game hấp dẫn hơn.
- d. Tuần 4: Hoàn thiện Trí tuệ Nhân tạo (AI), Cơ chế Chiến đấu và Nâng cấp Đồ họa (Polishing)**
- **Đa dạng hóa Hệ thống Kẻ địch (Enemy AI & Behaviors):** Kế thừa kiến trúc Enemy_Base từ Tuần 3, tuần này đã phát triển thành công một hệ sinh thái quái vật đa dạng với các hành vi (AI) độc lập:
 - **Hệ thống Đi tuần (Patrol Enemy):** Lập trình logic tuần tra trong phạm vi cố định (Patrol Distance). Tích hợp thành công Máy trạng thái (FSM) cho phép quái vật (ví dụ: Skeleton, Slime) tự động chuyển đổi mượt mà giữa hoạt ảnh đi bộ (walk) và tấn công (attack) khi va chạm với nhân vật chính.
 - **Hệ thống Rượt đuổi (Chase Enemy):** Xây dựng thuật toán Radar nhận diện mục tiêu. Kẻ địch (Orc) có khả năng tính toán khoảng cách trục X, Y để phát hiện người chơi, tăng tốc độ truy đuổi và thực hiện hành vi tấn công cận chiến sát thương cao.
 - **Hệ thống Tấn công Tầm xa (Ranged/Aiming Enemy):** Triển khai lớp nhân vật có khả năng ngắm bắn tự động dựa trên vị trí tương đối giữa tâm kẻ địch và tâm người chơi, triệt tiêu hoàn toàn lỗi "lệch tâm ngắm" (blind spot).
- **Xây dựng Hệ thống Đạn đạo (Projectile System)**
 - **Thuật toán Quỹ đạo:** Lập trình thành công loại đường đạn cốt lõi:
 - Đạn bay thẳng (Tia laser/Doi): Sử dụng định lý Pytago và chuẩn hóa Vector (Vector Normalization) để tính toán hướng bay chuẩn xác.
- **Tối ưu hóa Bộ nhớ (Memory Management):** Xây dựng vòng đời tự hủy cho Projectile (giới hạn tầm bay max_range hoặc va chạm địa hình), ngăn chặn rò rỉ bộ nhớ khi đạn bay ra khỏi màn hình.

- **Nâng cấp Trải nghiệm Thị giác (Visuals & Polish)**
 - Thay thế hoàn toàn các khối hình học cơ bản (placeholder) bằng tài nguyên đồ họa thực tế (Sprite Sheets).
 - **Xử lý Đồ họa Động:** Tích hợp kỹ thuật toán học để tính toán chính xác kích thước lưới ảnh (Grid) và độ lệch lề (Padding/Spacing). Ứng dụng kỹ thuật set_colorkey để khử nền đen tĩnh, đồng thời sử dụng transform.scale để đồng bộ kích thước (Scale Factor) của các thực thể khác nhau vào cùng một hệ quy chiếu bản đồ.
- **Hiệu ứng Vật lý & Toán học nâng cao:**
 - *Auto-Rotation:* Viết thuật toán xoay hình học (math.atan2) giúp các thực thể bay (như đạn dơi) tự động chĩa đầu theo hướng của Vector vận tốc.
 - *Levitation:* Áp dụng đồ thị hàm Lượng giác (math.sin) để tạo hiệu ứng lơ lửng, nhấp nhô nhịp nhàng trên không trung cho kẻ địch đặc biệt (Boss/Wizard), tạo cảm giác chân thực thay vì đứng im cứng nhắc.

2. Kết quả đạt được

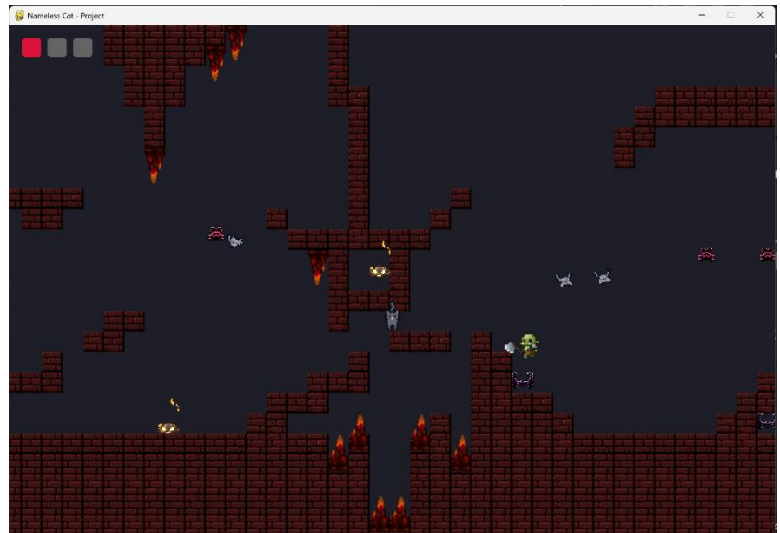
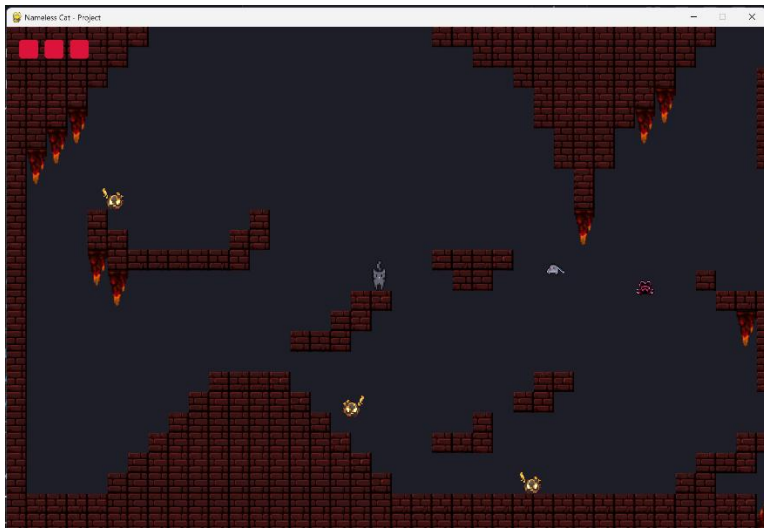
- **Xây dựng thành công Kiến trúc Lõi (Core Framework) chuẩn OOP**
 - **Đóng gói và Tái sử dụng:** Chuẩn hóa toàn bộ logic xử lý phức tạp (Vật lý, Kết xuất đồ họa, Vòng lặp Hoạt ảnh) thành một khung làm việc (framework) độc lập. Kiến trúc này tạo ra tính kế thừa (Inheritance) mạnh mẽ, cho phép khởi tạo hàng loạt thực thể (Player, Enemy, Projectile) một cách tối ưu, không bị trùng lặp mã nguồn.
- **Trạng Thái Hữu Hạn (FSM):** Triển khai thành công FSM ở hai cấp độ: Cấp độ Trò chơi (Start Menu, Playing, Game Over) và Cấp độ Thực thể (Idle, Walk, Attack, Chase), giúp điều phối luồng game mượt mà và logic.
- **Phát triển Engine Vật lý và Môi trường Tương tác (Interactive Testbed)**
- **Không gian Kiểm thử:** Tích hợp thành công môi trường tương tác thực tế ở tốc độ tiêu chuẩn 60 FPS. Đây là không gian testbed hoàn hảo để tinh chỉnh các thông số trọng lực, gia tốc, và ma sát.

- **Thuật toán Va chạm AABB:** Xử lý triệt để bài toán va chạm đa chiều (trục X, Y) với địa hình phức tạp từ file CSV, kết hợp thuật toán tính toán "hitbox thực" (loại bỏ transparent padding), đảm bảo nhân vật tương tác với môi trường vật lý với độ chính xác tuyệt đối (không đi xuyên tường, không lọt sàn).
- **Xây dựng Hệ sinh thái Trí tuệ Nhân tạo (Enemy AI & Behaviors)**
- Game không chỉ dừng lại ở các chương ngại vật tĩnh mà đã sở hữu một hệ thống AI đa dạng với 3 lớp hành vi độc lập:
 - **Patrol AI:** Kẻ địch biết tự động tuần tra trong cự ly giới hạn.
 - **Chase/Radar AI:** Kẻ địch nhận diện mục tiêu dựa trên khoảng cách (Vector X, Y) để tăng tốc độ truy đuổi.
 - **Ranged AI:** Kẻ địch tính toán quỹ đạo bắn tự động bằng Toán học ứng dụng (Vector Normalization, Định lý Pytago, Toán ném xiên Parabol).
- **Tối ưu hóa Bộ nhớ và Trải nghiệm Thị giác (Memory & Visual Polish)**
- **Trích xuất Khung hình Động (Dynamic Frame Extraction):** Viết thành công module xử lý Sprite Sheet tự động tính toán lưới ảnh (Grid), khắc phục triệt để lỗi rác dữ liệu ảnh (visual artifacts).
- **Quản lý Vòng đời Thực thể:** Tối ưu hóa bộ nhớ RAM bằng cơ chế tự hủy (Garbage Collection) cho các thực thể phát sinh liên tục như Đạn đạo (Projectiles) khi chúng va chạm hoặc bay quá tầm.
- **Hiệu ứng Toán học Nâng cao:** Áp dụng thuật toán Lượng giác (math.sin) tạo hiệu ứng lơ lửng, và Hàm Arctangent (math.atan2) để tự động xoay góc nhìn của vật thể theo vector vận tốc, mang lại trải nghiệm đồ họa sống động, chuyên nghiệp.

3. Tài liệu tham khảo

- Gemini Pro (tham khảo cách sử dụng thư viện pygame, các hàm sẵn có)
- LDtk Course – Youtuber: small indie (tham khảo cách thiết kế map)
- **Mã nguồn (Github):** https://github.com/Ngaostzy/project_DSA_simple-2D-Game_04-2026.git

4. Phụ lục 1: Giới thiệu (demo) kết quả



Mô phỏng thuật toán AABB và hệ sinh thái game

5. Phụ lục 2: docstring

! Phụ lục được trích xuất nhờ thư viện ast

```
# NAMELESS CAT PROJECT: SOURCE CODE DOCUMENTATION

## File: `settings.py`
---

## File: `ui.py`
---

### Class: `UI`
**Description:**
```text
Manages the Heads-Up Display (HUD) and graphical overlay states for the game.
```

#### Method: `__init__`
**Description:**
```text
Initializes viewport dimensions and typography resources for UI rendering.

Args:
 screen_width (int): The horizontal resolution of the display window.
 screen_height (int): The vertical resolution of the display window.
```
```

```

'''
#### Method: `draw_health_bar`
**Description:**
```text
Renders discrete hit-point indicators to represent the player's current vitality.

Args:
 screen (pygame.Surface): The primary display surface to render the HUD on.
 current_hp (int): The player's active health points.
 max_hp (int, optional): The maximum health capacity. Defaults to 3.
'''

Method: `draw_game_over`
Description:
```text
Projects a semi-transparent overlay and state-transition prompts for the end-game
sequence.

Args:
    screen (pygame.Surface): The primary display surface.
'''

### Class: `ImageButton`
**Description:**
```text
Represents an interactive graphical user interface (GUI) component with sprite
scaling.
'''

Method: `__init__`
Description:
```text
Initializes the button's visual representation and spatial bounding box.

Args:
    x (float): The x-coordinate for the center of the button.
    y (float): The y-coordinate for the center of the button.
    image (pygame.Surface): The base graphical asset for the button.
    scale (float, optional): The uniform scaling multiplier. Defaults to 1.0.
'''

#### Method: `draw`
**Description:**
```text

```

```

Renders the button sprite onto the designated display surface.

Args:
 screen (pygame.Surface): The primary rendering surface.
 ...

Method: `is_clicked`
Description:
```text
Evaluates pointer-based interaction within the button's collision domain.

Args:
    event (pygame.event.Event): The input event queued by Pygame.

Returns:
    bool: True if a left mouse click intersects the bounding box, False
    otherwise.
    ...

## File: `camera.py`
---
### Class: `Camera`
**Description:**
```text
Manage camera scrolling based on a target within level boundaries.
 ...

Method: `__init__`
Description:
```text
Initialize the camera with screen dimensions.

Args:
    screen_width (int): Width of the display screen in pixels.
    screen_height (int): Height of the display screen in pixels.
    ...

#### Method: `get_target_center`
**Description:**
```text
Get the center position of a target object.

Args:
 target (object): Object whose position is determined from rect,

```

```

 hitbox, collision_rect, x/y/width/height, or x/y attributes.

Returns:
 tuple[int, int]: The x and y coordinates of the target center.

Raises:
 AttributeError: If the target does not contain supported position attributes.
 ...

Method: `update`
Description:
```text
Update camera scrolling to follow a target within level boundaries.

Args:
    target (object): Object followed by the camera.
    level_width (int): Total width of the level in pixels.
    level_height (int): Total height of the level in pixels.
    ...

## File: `level.py`
---
#### Class: `LEVEL`
**Description:**
```text
Load, parse, and render a 2D level from an LDtk map file.
 ...

Method: `__init__`
Description:
```text
Initialize the level data, assets, and parse the LDtk file.

Args:
    ldtk_filepath (str): Path to the LDtk JSON file used to load the level.
    ...

#### Method: `parse_ldtk`
**Description:**
```text
Parse an LDtk JSON file and extract level geometry and entities.

Args:
 filepath (str): Path to the LDtk JSON file.

```

```

Raises:
 FileNotFoundError: If the LDtk file does not exist.
 RuntimeError: If the file cannot be read, contains no levels,
 or has invalid map dimensions.
'''

Method: `_parse_blocks`
Description:
'''text
Parse the block layer into collision rectangles and renderable tiles.

Args:
 layer (dict): LDtk IntGrid layer containing block and tile data.

Raises:
 RuntimeError: If the block layer grid width is invalid.
'''

Method: `_parse_entities`
Description:
'''text
Parse entity spawn points from an LDtk entity layer.

Args:
 layer (dict): LDtk Entities layer containing player, portal,
 and enemy spawn data.
'''

Method: `_render`
Description:
'''text
Render the level surface to the screen using camera scroll offsets.

Args:
 screen (pygame.Surface): Target display surface.
 scroll_x (int): Horizontal camera scroll offset.
 scroll_y (int): Vertical camera scroll offset.

Raises:
 RuntimeError: If the level surface has not been created.
'''

File: `entity.py`

```

```

Class: `ENTITY`
Description:
```text
Represent a base movable and interactive game entity.

This class provides common attributes and methods for position, velocity,
animation, rendering, and tile-based collision handling.
```

Method: `__init__`
Description:
```text
Initialize the entity position, size, motion, and animation state.

Args:
    x (float): Initial x-coordinate of the entity.
    y (float): Initial y-coordinate of the entity.
    width (int): Width of the entity collision box.
    height (int): Height of the entity collision box.
```

Method: `extract_frames`
Description:
```text
Extract a sequence of animation frames from a sprite sheet row.

Args:
    row (int): Row index in the sprite sheet.
    num_frames (int): Number of frames to extract.
    frame_size (tuple[int, int]): Width and height of each frame.

Returns:
    list[pygame.Surface]: List of processed animation frames.
```

Method: `extract_custom_frames`
Description:
```text
Extract specific animation frames from sprite sheet coordinates.

Args:
    frame_coords (list[tuple[int, int]]): List of frame positions as
        row and column pairs.
    frame_size (tuple[int, int]): Width and height of each frame.

```

```

Returns:
    list[pygame.Surface]: List of processed animation frames.
    ...

#### Method: `_process_frame`
**Description:**
    ``text
Crop, trim, and scale a frame from the sprite sheet.

Args:
    x (int): Source x-coordinate of the frame.
    y (int): Source y-coordinate of the frame.
    w (int): Width of the frame.
    h (int): Height of the frame.

Returns:
    pygame.Surface | None: Processed frame surface, or None if processing fails.
    ...

#### Method: `_update_animations`
**Description:**
    ``text
Update the current animation frame based on the entity state.

Advances the animation timer using the configured speed and loops the
animation when it reaches the final frame.
    ...

#### Method: `_render`
**Description:**
    ``text
Render the entity image to the screen with camera offset.

Args:
    screen (pygame.Surface): Target surface used for rendering.
    camera_x (float): Horizontal camera offset.
    camera_y (float): Vertical camera offset.
    ...

#### Method: `_get_nearby_platforms`
**Description:**
    ``text
Retrieve nearby platforms using spatial grid coordinates.

```

```

Args:
    spatial_platforms (dict[tuple[int, int], pygame.Rect]): Dictionary
        mapping tile coordinates to platform rectangles.
    tile_size (int): Size of each tile in pixels.

Returns:
    list[pygame.Rect]: Platforms located near the entity.
    ...

#### Method: `handle_vertical_collision`
**Description:**
```text
Resolve vertical AABB collisions with nearby platforms.

Updates the vertical position, resets vertical velocity on collision,
and sets the grounded state when the entity lands on a platform.

Args:
 spatial_platforms (dict[tuple[int, int], pygame.Rect]): Dictionary
 mapping tile coordinates to platform rectangles.
 ...

Method: `handle_horizontal_collision`
Description:
```text
Resolve horizontal AABB collisions with nearby platforms.

Updates the horizontal position and prevents the entity from passing
through platform boundaries.

Args:
    spatial_platforms (dict[tuple[int, int], pygame.Rect]): Dictionary
        mapping tile coordinates to platform rectangles.
    ...

## File: `player.py`
---
#### Class: `PLAYER`
**Description:**
```text
Represent the main controllable player character.

The player handles keyboard input, movement physics, animation updates,
health management, damage invincibility, and rendering.

```

```

'''
Method: `__init__`
Description:
'''text
Initialize the player entity, movement attributes, and animations.

Args:
 x (float): Initial x-coordinate of the player in world space.
 y (float): Initial y-coordinate of the player in world space.
 width (int): Width of the player's collision box.
 height (int): Height of the player's collision box.
'''

Method: `update`
Description:
'''text
Update player input, movement physics, invincibility, and animation.

Applies gravity, limits falling speed, processes keyboard movement,
handles jumping, and advances the current animation frame.
'''

Method: `take_damage`
Description:
'''text
Apply damage to the player and activate temporary invincibility.

Args:
 amount (int): Amount of health points to subtract from the player.
'''

Method: `render`
Description:
'''text
Render the player sprite with camera offset and invincibility effect.

Args:
 screen (pygame.Surface): Target surface used for rendering.
 camera_x (float): Horizontal camera scroll offset.
 camera_y (float): Vertical camera scroll offset.
'''

File: `portal.py`

```

```

Class: `PORTAL`
Description:
```text
Represents a portal entity for spatial transitions within the game environment.
```

Method: `__init__`
Description:
```text
Initializes the portal's spatial dimensions and visual properties.

Args:
    x (float): The x-coordinate of the portal's top-left corner.
    y (float): The y-coordinate of the portal's top-left corner.
    width (int, optional): The bounding box width in pixels. Defaults to 64.
    height (int, optional): The bounding box height in pixels. Defaults to 64.
```

Method: `check_collision`
Description:
```text
Evaluates Axis-Aligned Bounding Box (AABB) collision between the portal and the
player.

Args:
    player (object): The player entity instance. Must possess 'x', 'y', 'width',
and 'height' attributes.

Returns:
    bool: True if bounding boxes intersect, False otherwise.
```

Method: `render`
Description:
```text
Renders the portal onto the display surface with a time-based oscillating alpha
effect.

Args:
    screen (pygame.Surface): The primary display surface to render the portal on.
    scroll_x (float): The current horizontal camera offset.
    scroll_y (float): The current vertical camera offset.
```

```

```

File: `projectile.py`

Class: `PROJECTILE`
Description:
```text
Represents a dynamic projectile entity with kinematic behavior and animated
sprites.
```

Method: `__init__`
Description:
```text
Initializes the projectile's physical properties, payload, and visual assets.

Args:
    x (float): The initial x-coordinate of the projectile.
    y (float): The initial y-coordinate of the projectile.
    vx (float): Horizontal velocity component.
    vy (float): Vertical velocity component.
    damage (int, optional): The damage payload delivered upon impact. Defaults to
1.
    max_range (int, optional): Maximum travel distance before despawning.
Defaults to 400.
```

Method: `update`
Description:
```text
Processes the projectile's kinematics, sprite rotation, and environmental
collision.

Args:
    tiles (dict/set): A collection of static grid coordinates representing
obstacles.
```

Method: `render`
Description:
```text
Projects the visual representation onto the 2D viewport.

Args:
    screen (pygame.Surface): The primary display surface.
    camera_x (float): The current horizontal camera translation offset.
    camera_y (float): The current vertical camera translation offset.

```

```

'''
## File: `spikes.py`
---
### Class: `SPIKE`
**Description:**
```text
Represents a static environmental hazard with orientation-based collision
mechanics.
'''

Method: `__init__`
Description:
```text
Initializes spatial parameters, damage payload, and graphic assets for the
hazard.

Args:
    x (float): The x-coordinate of the tile's top-left origin.
    y (float): The y-coordinate of the tile's top-left origin.
    width (int, optional): The visual width of the spike tile. Defaults to 32.
    height (int, optional): The visual height of the spike tile. Defaults to 32.
    orientation (str, optional): The directional alignment ('up' or 'down').
Defaults to 'up'.
'''

#### Method: `_draw_fallback`
**Description:**
```text
Generates a procedural polygonal representation if the image asset is
unavailable.
'''

Method: `get_hitbox`
Description:
```text
Computes the precise Axis-Aligned Bounding Box (AABB) taking into account
geometric padding.

Returns:
    pygame.Rect: The internal collision boundary, strictly smaller than the
visual tile.
'''

```

```

#### Method: `check_collision`
**Description:**
```text
Evaluates spatial intersection with a given entity and triggers damage states.

Args:
 player (object): The entity instance subject to collision and damage
 resolution.

Returns:
 bool: True if a hazardous collision occurred, False otherwise.
```

#### Method: `render`
**Description:**
```text
Projects the sprite onto the display surface relative to the viewport
translation.

Args:
 screen (pygame.Surface): The primary rendering surface.
 camera_x (float, optional): Viewport horizontal translation. Defaults to 0.0.
 camera_y (float, optional): Viewport vertical translation. Defaults to 0.0.
```

## File: `chase_enemy.py`
---
### Class: `CHASE_ENEMY`
**Description:**
```text
Represents an advanced hostile entity featuring line-of-sight detection, state-
driven chasing, and melee attack mechanics.
```

#### Method: `__init__`
**Description:**
```text
Initializes the entity's kinematic base, perception thresholds, and state-
dependent animation assets.

Args:
 x (float): Initial x-coordinate and patrol anchor.
 y (float): Initial y-coordinate.
 width (int): Bounding box width.

```

```

 height (int): Bounding box height.
 patrol_distance (float, optional): Maximum lateral patrol deviation. Defaults
to 300.0.
 hp (int, optional): Maximum health points. Defaults to 5.
 damage (int, optional): Damage payload delivered upon attack phase. Defaults
to 2.
'''

Method: `check_player_interactions`
Description:
'''text
Evaluates spatial intersections and line-of-sight heuristics to transition
between patrol, chase, and attack states.

Args:
 player (object): The player entity subject to vision tracking and collision
detection.
'''

Method: `update`
Description:
'''text
Advances the entity's kinematic logic, state-machine behavioral branching, and
sprite animations per logical frame.
'''

File: `enemy_base.py`

Class: `ENEMY_BASE`
Description:
'''text
Abstract base class for hostile entities, managing kinematics, spatial awareness,
and combat resolution.
'''

Method: `__init__`
Description:
'''text
Initializes foundational attributes for the enemy entity.

Args:
 x (float): The initial x-coordinate of the entity.
 y (float): The initial y-coordinate of the entity.
 width (int): Bounding box width.

```

```

 height (int): Bounding box height.
 hp (int, optional): Maximum health points. Defaults to 3.
 damage (int, optional): Damage payload delivered upon contact. Defaults to 1.
 ...

Method: `check_player_interactions`
Description:
```text
Evaluates AABB intersection with the player, applying damage and kinetic
knockback.

Args:
    player (object): The player entity instance to check against.
    ...

#### Method: `handle_horizontal_collision`
**Description:**
```text
Resolves lateral displacement against environmental constraints via raycasting
and edge detection.

Args:
 spatial_platforms (list/dict): A collection of platform bounds for spatial
queries.
 ...

Method: `apply_gravity`
Description:
```text
Applies continuous downward acceleration bounded by terminal velocity.
    ...

#### Method: `update`
**Description:**
```text
Advances the kinematic state of the entity per logical frame.
 ...

File: `patrol_enemy.py`

Class: `PATROL_ENEMY`
Description:
```text

```

```

Represents a hostile entity with an automated, distance-bounded oscillating
patrol routine.
'''

#### Method: `__init__`
**Description:**
```text
Initializes the patrolling entity's kinematic properties, spatial anchors, and
visual assets.

Args:
 x (float): The initial x-coordinate, serving as the central anchor for the
patrol route.
 y (float): The initial y-coordinate.
 width (int): Bounding box width.
 height (int): Bounding box height.
 patrol_distance (float, optional): The maximum lateral deviation from the
starting anchor. Defaults to 150.0.
 hp (int, optional): Maximum health points. Defaults to 4.
 damage (int, optional): Damage payload delivered upon contact. Defaults to 2.
'''

Method: `update`
Description:
```text
Processes the entity's frame-by-frame state, strictly enforcing spatial patrol
boundaries.
'''

## File: `ranged_aiming.py`
---
#### Class: `AIMING_ENEMY`
**Description:**
```text
Represents an aerial hostile entity with sinusoidal hovering mechanics and
vector-based projectile targeting.
'''

Method: `__init__`
Description:
```text
Initializes combat parameters, floating constraints, and projectile reservoirs.

Args:

```

```

    x (float): Initial x-coordinate.
    y (float): Initial y-coordinate.
    width (int): Bounding box width.
    height (int): Bounding box height.
    hp (int, optional): Maximum health points. Defaults to 2.
    damage (int, optional): Contact damage payload. Defaults to 1.
    ...

#### Method: `update`
**Description:**
```text
Computes sinusoidal vertical displacement and advances the kinematic state of
active projectiles.

Args:
 tiles (list, optional): Environmental grid segments for projectile collision
resolution. Defaults to None.
 ...

Method: `check_player_interactions`
Description:
```text
Resolves projectile-entity intersections and evaluates line-of-sight vector
magnitude for attack triggering.

Args:
    player (object): The target entity for collision and targeting heuristics.
    ...

#### Method: `shoot`
**Description:**
```text
Normalizes the directional vector towards the target to instantiate and propel a
projectile.

Args:
 dist_x (float): The horizontal scalar distance to the target.
 dist_y (float): The vertical scalar distance to the target.
 ...

Method: `render`
Description:
```text
Projects the entity and its localized projectile pool onto the viewport.

```

```

Args:
    screen (pygame.Surface): The primary display surface.
    camera_x (float): Viewport horizontal translation.
    camera_y (float): Viewport vertical translation.
    ...

## File: `ranged_horizontal.py`
---
### Class: `RANGED_HORIZONTAL`
**Description:**
```text
Represents a ranged hostile entity capable of horizontal projectile emission
based on proximity-triggered aggression.
```

#### Method: `__init__`
**Description:**
```text
Initializes combat attributes, projectile reservoirs, and temporal cooldown
parameters.

Args:
 x (float): Initial x-coordinate.
 y (float): Initial y-coordinate.
 width (int): Bounding box width.
 height (int): Bounding box height.
 hp (int, optional): Maximum health points. Defaults to 3.
 damage (int, optional): Contact damage payload. Defaults to 1.
 ...

Method: `shoot`
Description:
```text
Instantiates and propels a projectile along the horizontal axis aligned with the
entity's current orientation.
```

Method: `update`
Description:
```text
Advances the kinematic state of the entity and processes spatial updates for all
active projectiles.

Args:

```

```

    tiles (list, optional): Environmental grid constraints for projectile
collision. Defaults to None.
'''

#### Method: `check_player_interactions`
**Description:**
```text
Evaluates projectile intersections and triggers temporal-based firing sequences
within specific spatial boundaries.

Args:
 player (object): The target entity for collision resolution and proximity
detection.
'''

Method: `render`
Description:
```text
Projects the entity and its active projectiles onto the 2D viewport.

Args:
    screen (pygame.Surface): The primary display surface.
    camera_x (float): Viewport horizontal translation.
    camera_y (float): Viewport vertical translation.
'''

```